

Ejada Egypt Summer Internship Program 2026

Week 3 Technical Documentation

Prepared by: Yara Adel Khedr

github link: <https://github.com/Yara-Khedr/ejada-intern-terraform>

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Environment	3
2	Architecture Diagram	4
3	Network Architecture	5
3.1	VCN and CIDR	5
3.2	Public Subnet	5
3.3	Private Subnet	5
3.4	Gateways	5
3.5	Route Tables	5
4	Security Design	6
4.1	Security Lists vs. NSGs	6
4.2	Public Security List	6
4.3	Private Security List	6
5	Load Balancer	7
6	OKE Cluster and Node Pool	8
6.1	Cluster	8
6.2	Node Pool	8
6.3	Cluster Access Model	9
7	Container Registry (OCIR)	10
7.1	Image Build	10
7.2	Push to OCIR	10
7.3	Cluster Pull Authentication	11
8	Application Deployment and Storage	12
8.1	Namespace	12
8.2	Deployment	12
8.3	Persistent Storage	12
9	Terraform Implementation	14
9.1	File Structure	14
9.2	Variables, Locals, Data Sources	14
9.3	Remote State	14
10	Issues Encountered	16
10.1	Child Module Provider Resolution	16
10.2	Worker Node Image Selection	16
10.3	Node Registration Timeout	16
10.4	OCIR Authentication	16
10.5	Persistent Volume Multi-Attach Conflict	16
10.6	Local Client Connectivity to the Cluster Endpoint	16
11	Testing and Validation	17
11.1	Network Topology	17
11.2	Cluster and Node Health	17

11.3 Application and Storage	17
--	----

1 Introduction

1.1 Purpose

This document covers the design, deployment, and validation of the Week 3 environment: a network-segmented OCI environment hosting an Oracle Kubernetes Engine cluster, built with a reusable Terraform subnet module, running a containerized application pulled from a private OCI Container Registry repository and exposed publicly through an OCI Load Balancer with persistent block storage.

1.2 Environment

Attribute	Value
Tenancy	ociejada
Region	me-jeddah-1
Compartment	intern-17-yara-khedr-cmp
Terraform / OCI CLI	1.15.8 / 3.90.1
OCI Provider	oracle/oci 8.27.0
Kubernetes version	v1.34.2
Terraform Code (Week 3)	https://github.com/Yara-Khedr/ejada-intern-terraform/tree/main/week3

Table 1: Environment summary

Resources follow the naming pattern `Yara-W3-<ResourceType>`, marking them as Week 3 builds.

2 Architecture Diagram

Full architecture – VCN, subnets, gateways, OKE cluster, node pool, OCIR, and Load Balancer.

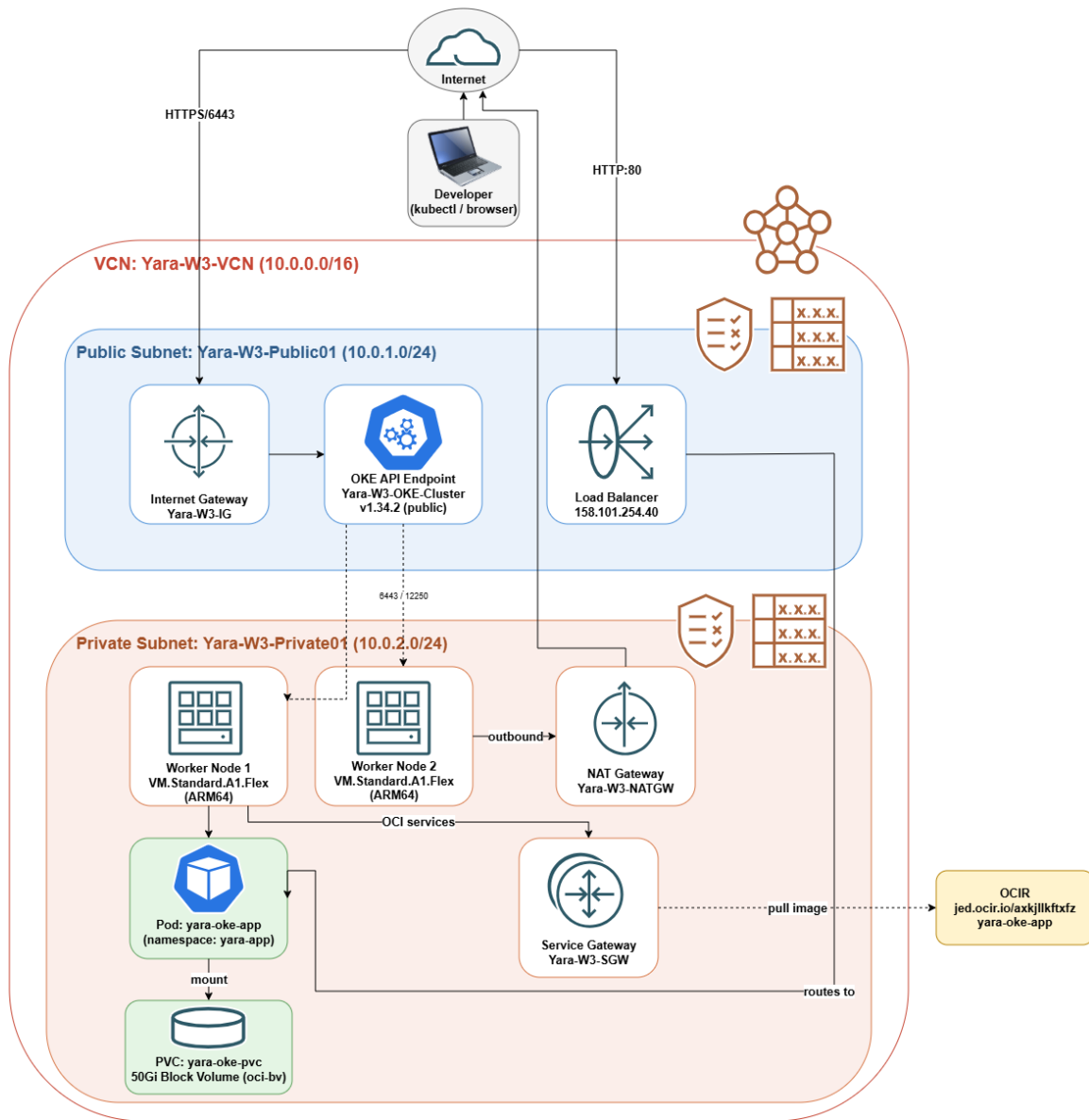


Figure 1: Week 3 OCI and OKE architecture

3 Network Architecture

3.1 VCN and CIDR

Yara-W3-VCN uses 10.0.0.0/16, split into two /24 subnets.

Network	CIDR	Purpose
Yara-W3-VCN	10.0.0.0/16	VCN
Yara-W3-Public01	10.0.1.0/24	Public tier – OKE endpoint, Load Balancer
Yara-W3-Private01	10.0.2.0/24	Private tier – worker nodes, pods

Table 2: CIDR allocation

3.2 Public Subnet

Hosts the OKE API endpoint (public) and the auto-provisioned Load Balancer. Routed via the public route table to the Internet Gateway.

3.3 Private Subnet

Hosts the two OKE worker nodes and all pods (VCN-native pod networking, using the same subnet for both). No public IPs. Inbound traffic accepted only from within the VCN; outbound via NAT Gateway; OCI platform service access (including OCIR) via Service Gateway.

3.4 Gateways

Gateway		Role
Yara-W3-IG (Internet GW)	Public subnet in/out internet access	
Yara-W3-NATGW (NAT GW)	Outbound-only internet access for private subnet	
Yara-W3-SGW (Service GW)	Private access to OCI services (incl. OCIR) over Oracle backbone	

Table 3: Gateway roles

3.5 Route Tables

Route Table	Destination	Target
Public route table	0.0.0.0/0	Internet Gateway
Private route table	0.0.0.0/0	NAT Gateway
	All JED Services in Oracle Services Network	Service Gateway

Table 4: Route rules by subnet

4 Security Design

4.1 Security Lists vs. NSGs

Security Lists were used instead of NSGs: each subnet has one clear role (public endpoint/LB tier vs. private compute tier), so subnet-level rules are sufficient. NSGs suit cases where instances in the *same* subnet need different rules, which does not apply here.

4.2 Public Security List

Direction	Source/Dest.	Protocol	Port	Purpose
Ingress	0.0.0.0/0	TCP	80	Application HTTP traffic
Ingress	10.0.2.0/24	TCP	6443	Kubernetes API (node registration)
Ingress	10.0.2.0/24	TCP	12250	Kubelet-to-control-plane
Ingress	10.0.2.0/24	ICMP	3,4	Path MTU Discovery
Ingress	<i>developer IP</i> /32	TCP	6443	Local <code>kubectl</code> access
Egress	0.0.0.0/0	All	All	

Table 5: Public subnet security rules

4.3 Private Security List

Direction	Source/Dest.	Protocol	Port	Purpose
Ingress	10.0.0.0/16	All	All	Intra-VCN traffic
Ingress	0.0.0.0/0	ICMP	3,4	Path MTU Discovery
Egress	0.0.0.0/0	All	All	

Table 6: Private subnet security rules

Design Note

Worker nodes in the private subnet initially could not reach the Kubernetes API endpoint on the ports kubelet requires, causing node registration to time out. TCP/6443 and TCP/12250 ingress from the private subnet CIDR were added to the public subnet's security list to resolve this – see Section 10.3.

5 Load Balancer

A Kubernetes **Service** of type **LoadBalancer** causes OKE to automatically provision a real OCI Load Balancer in the public subnet (per `service_lb_subnet_ids` in the cluster configuration), exposing the application at a public IP. Although the Service manifest is applied as part of the application deployment (Section 8), the Load Balancer itself is a network-facing resource and is documented here alongside the rest of the network design.

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: yara-oke-app-service
  namespace: yara-app
spec:
  type: LoadBalancer
  selector:
    app: yara-oke-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

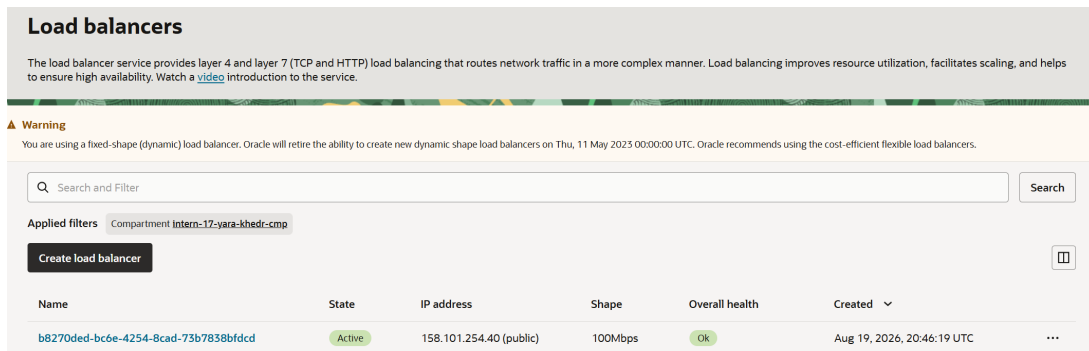


Figure 2: Auto-provisioned Load Balancer – ACTIVE

Browsing the public IP returns the application, confirming the full path – Load Balancer to pod to persistent volume – works end to end.

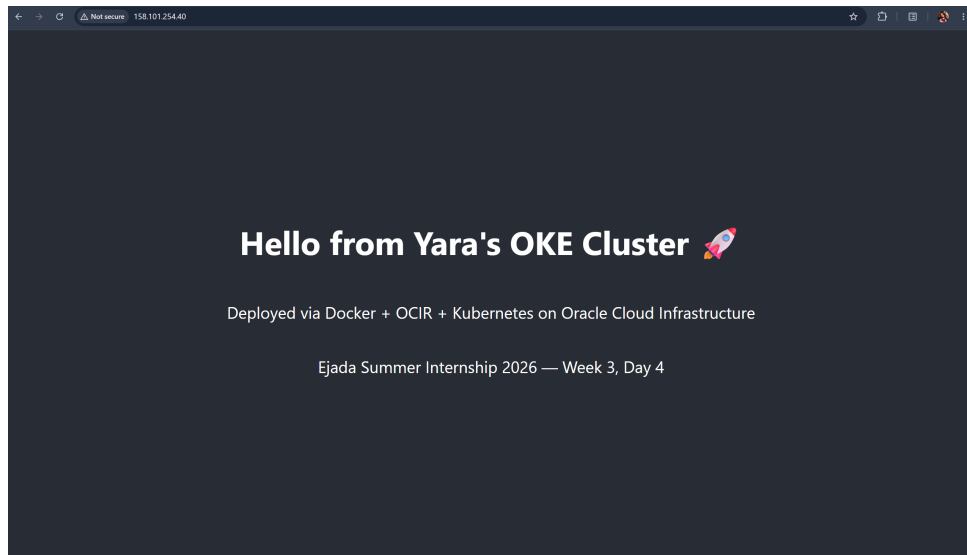


Figure 3: End-to-end verification via <http://158.101.254.40>

6 OKE Cluster and Node Pool

6.1 Cluster

Yara-W3-OKE-Cluster, Kubernetes v1.34.2, CNI type OCI_VCN_IP_NATIVE, public endpoint bound to the public subnet, Load Balancer subnet also bound to the public subnet.

 A screenshot of the Oracle Cloud Infrastructure (OCI) console showing the details of the "Yara-W3-OKE-Cluster". The cluster is in an "Active" state. The interface is divided into two main sections: "Cluster details" and "Network information".

Cluster details		Network information	
Cluster status	Active	VCN name	Yara-W3-VCN
Node pools	1	VCN ID	...niakpivyfi47gwiz5qhwrlftgxiyl3x7uptk4rjqelicia Copy
Cluster ID	...zzio6Su3unwrf6j5z4f3ynf3c2nqn7ybn2q Copy	Compartment	intern-17-yara-khedr-cmp
Compartment	intern-17-yara-khedr-cmp	Services CIDR	10.96.0.0/16
Launched	Aug 19, 2026, 18:12:10 UTC	Network type	OCI_VCN_IP_NATIVE
Created	-	Kubernetes API endpoint subnet	Yara-W3-Public01-subnet
Kubernetes version	v1.34.2 Upgrade recommended Upgrade	Kubernetes API private IPv4 endpoint	10.0.1.83:6443 Copy
Secrets encryption key	Oracle-managed	Kubernetes API IPv6 endpoint	Not assigned
Cluster type	Basic	Kubernetes API public endpoint	193.122.87.220:6443 Copy
Cluster credentials expire on	8/19/2031 Start Rotation	Network security group	Not enabled
Complete credential rotation on	—	Service LB subnet 1	Yara-W3-Public01-subnet

Figure 4: Yara-W3-OKE-Cluster – ACTIVE

6.2 Node Pool

Yara-W3-NodePool01: shape VM.Standard.A1.Flex (Ampere ARM64, Always Free), 1 OCPU / 8 GB per node, 2 nodes (2 OCPU / 16 GB total, within the 4 OCPU / 24 GB Always Free cap), deployed into the private subnet via VCN-native pod networking (`pod_subnet_ids` nested inside `node_config_details`).

Worker node images must be OKE-optimized, obtained via `oci ce node-pool-options get` and matched exactly to the cluster's Kubernetes version – a generic Oracle Linux image of the

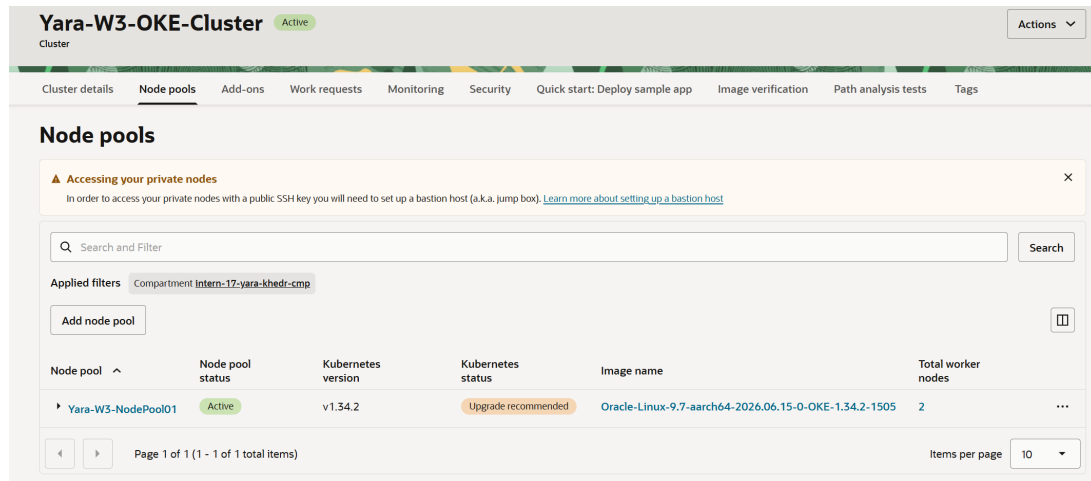


Figure 5: Yara-W3-NodePool01 – two ACTIVE nodes

correct OS/architecture is rejected.

6.3 Cluster Access Model

`kubectl` is configured via `oci ce cluster create-kubeconfig`, using the cluster's public API endpoint. Local access from outside the VCN required an explicit security list rule; this differs from the Week 2 model, where private-tier access ran entirely through a Jump VM inside the VCN.

kubectl get nodes

```
NAME STATUS ROLES AGE VERSION
10.0.2.141 Ready node 38m v1.34.2
10.0.2.180 Ready node 38m v1.34.2
```

7 Container Registry (OCIR)

The application is packaged as a Docker image and stored in a private OCIR repository, `yara-oke-app`, under namespace `axkjllkftxfz`.

7.1 Image Build

The application is a React single-page app, containerized with a multi-stage Dockerfile: a Node.js build stage compiles static assets, and a minimal NGINX stage serves them.

Dockerfile

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Because the node pool runs on ARM64 hardware, the image was cross-compiled using Docker Buildx rather than built natively on the (AMD64) development machine:

docker buildx build

```
$ docker buildx build --platform linux/arm64 -t yara-oke-app:latest --load .
...
[+] Building 682.4s (14/14) FINISHED
```

7.2 Push to OCIR

docker tag / docker push

```
$ docker tag yara-oke-app:latest jed.ocir.io/axkjllkftxfz/yara-oke-app:latest
$ docker push jed.ocir.io/axkjllkftxfz/yara-oke-app:latest
latest: digest: sha256:2
a093c9bb3c49a781c6f2a5801e2a6e579fc8b4ba250008d663447167fb5b4a4 size: 856
```

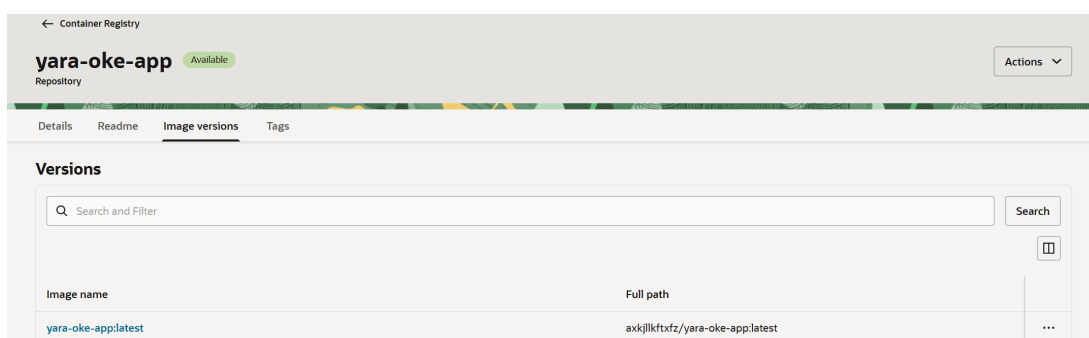


Figure 6: yara-oke-app repository with pushed image

7.3 Cluster Pull Authentication

The repository is private, so the cluster requires its own credential – separate from the credential used to push locally – to pull the image. This was provided as a Kubernetes **docker-registry** Secret, authenticated with a short-lived OCIR bearer token obtained from the existing OCI CLI session, and referenced in the Deployment via **imagePullSecrets**.

8 Application Deployment and Storage

8.1 Namespace

A dedicated namespace, `yara-app`, isolates all application resources from the cluster default.

8.2 Deployment

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: yara-oke-app
  namespace: yara-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: yara-oke-app
  template:
    metadata:
      labels:
        app: yara-oke-app
    spec:
      imagePullSecrets:
        - name: ocir-secret
      volumes:
        - name: yara-oke-storage
          persistentVolumeClaim:
            claimName: yara-oke-pvc
      containers:
        - name: yara-oke-app
          image: jed.ocir.io/axkjllkftxfz/yara-oke-app:latest
          ports:
            - containerPort: 80
          volumeMounts:
            - name: yara-oke-storage
              mountPath: /usr/share/nginx/html/data
```

8.3 Persistent Storage

A `PersistentVolumeClaim` requests a 50 GiB OCI Block Volume via the cluster's default `oci-bv` `StorageClass`.

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: yara-oke-pvc
  namespace: yara-app
spec:
  storageClassName: "oci-bv"
  accessModes:
    - ReadWriteOnce
```

```
resources:
  requests:
    storage: 50Gi
```

`ReadWriteOnce` permits the volume to attach to only one node at a time; running two replicas against this shared volume produced a `Multi-Attach` scheduling failure, resolved by running a single replica – correct both for the storage type's constraints and for a stateless single-page application.

Persistence was verified by writing a test file into the mounted volume, deleting the pod entirely, and confirming the file was still present on the replacement pod created automatically by the Deployment controller.

Persistence verification – write

```
$ kubectl exec -n yara-app yara-oke-app-564858d65c-bcjfb -- sh -c \
    "echo 'Hello from Week3 Day4 persistent volume test' > /usr/share/nginx/html/
    data/test.txt"

$ kubectl exec -n yara-app yara-oke-app-564858d65c-bcjfb -- \
    cat /usr/share/nginx/html/data/test.txt
Hello from Week3 Day4 persistent volume test

$ kubectl delete pod yara-oke-app-564858d65c-bcjfb -n yara-app
pod "yara-oke-app-564858d65c-bcjfb" deleted from yara-app namespace

$ kubectl get pods -n yara-app
NAME READY STATUS RESTARTS AGE
yara-oke-app-564858d65c-2l5jm 0/1 ContainerCreating 0 12s

$ kubectl get pods -n yara-app
NAME READY STATUS RESTARTS AGE
yara-oke-app-564858d65c-2l5jm 1/1 Running 0 80s

$ kubectl exec -n yara-app yara-oke-app-564858d65c-2l5jm -- \
    cat /usr/share/nginx/html/data/test.txt
Hello from Week3 Day4 persistent volume test
```

9 Terraform Implementation

9.1 File Structure

Infrastructure is defined as two reusable child modules, called from a root configuration, each with its own provider requirements.

File / Directory	Purpose
providers.tf	OCI provider config, remote backend
variables.tf	/ Inputs
terraform.tfvars	
data.tf	Data source lookups (OCI services)
locals.tf	Derived values (route rules)
main.tf	Module orchestration
outputs.tf	Outputs
modules/subnet/	Reusable subnet module (route table, security list, subnet, flow log)
modules/oke/	OKE cluster and node pool module

Table 7: File structure – week3/

9.2 Variables, Locals, Data Sources

Values that vary by environment – compartment ID, subnet CIDRs, node pool sizing, OKE cluster configuration – are pulled from variables. Locals hold values derived from data sources: the Oracle Services Network’s service ID and CIDR, used by both the Service Gateway resource and the private subnet’s route rules.

9.3 Remote State

State is stored using Terraform’s native `oci` backend, pointing at an Object Storage bucket (`yara-w3-tfstate`) created manually and deliberately outside Terraform’s own management, since a resource cannot manage the state file describing its own existence.

To confirm the backend is genuinely active – rather than Terraform silently falling back to a local state file – three checks were run: that Terraform can read the full resource set from the backend, that the corresponding object physically exists in the OCI bucket with a matching size and a recent modification time, and that the local `.terraform` directory holds only a small backend pointer rather than a full copy of state.

terraform state list

```
$ terraform state list
data.oci_core_services.all_services
oci_core_internet_gateway.week3_ig
oci_core_nat_gateway.week3_natgw
oci_core_service_gateway.week3_sgw
oci_core_vcn.week3_vcn
oci_logging_log_group.week3_log_group
module.oke.oci_containerengine_cluster.oke_cluster
module.oke.oci_containerengine_node_pool.oke_node_pool
module.private_subnet.oci_core_route_table.subnet_route_table
module.private_subnet.oci_core_security_list.subnet_security_list
module.private_subnet.oci_core_subnet.subnet
```

```
module.subnet.oci_core_route_table.subnet_route_table
module.subnet.oci_core_security_list.subnet_security_list
module.subnet.oci_core_subnet.subnet
```

oci os object list – state bucket

```
$ oci os object list --bucket-name yara-w3-tfstate --namespace axkjl1kftxfz
{
  "data": [
    {
      "name": "week3/terraform.tfstate",
      "size": 38190,
      "storage-tier": "Standard",
      "time-created": "2026-08-19T16:35:04.968000+00:00",
      "time-modified": "2026-08-19T19:00:23.510000+00:00"
    }
  ]
}
```

local state pointer

```
$ dir .terraform\terraform.tfstate
08/19/2026 07:09 PM 673 terraform.tfstate
```

The local file's size (673 bytes) is two orders of magnitude smaller than the object in the bucket (38,190 bytes) and holds only backend connection metadata, confirming that all real resource state is being read from and written to Object Storage rather than tracked locally.

10 Issues Encountered

10.1 Child Module Provider Resolution

Cause: Child modules must declare their own `required_providers` block; this is not inherited from the root module. Without it, Terraform silently defaults to the legacy `hashicorp/oci` provider.

Fix: Added `providers.tf` to each child module and reinitialized from a clean `.terraform/` directory.

10.2 Worker Node Image Selection

Cause: Invalid `nodeSourceDetails.imageId`: Node image not supported, using a generic Oracle Linux image matching the correct OS version and architecture – OKE requires images specifically tagged as OKE-optimized.

Fix: Selected the OKE-optimized image via `oci ce node-pool-options get`, matched exactly to the cluster's Kubernetes version.

10.3 Node Registration Timeout

Cause: The public subnet's security list allowed only TCP/80 inbound; nothing permitted worker nodes in the private subnet to reach the Kubernetes API endpoint on the ports kubelet requires. Node pool creation succeeded, but nodes never registered (25-minute timeout).

Fix: Added ingress rules for TCP/6443 and TCP/12250 from the private subnet CIDR to the public subnet's security list. Registration then completed successfully.

10.4 OCIR Authentication

Cause: `docker login` to OCIR repeatedly returned `Unauthorized` using a Console-generated long-lived Auth Token, despite correct username formatting and confirmed IAM permissions (verified independently via a successful `oci artifacts container repository create` call).

Fix: Authenticated instead using a short-lived OCIR bearer token generated directly from the existing OCI CLI session (`oci container-registry access-token get`), using the literal username `BEARER.TOKEN`. This succeeded immediately and was reused for both the local `docker push` and the Kubernetes `imagePullSecret`.

10.5 Persistent Volume Multi-Attach Conflict

Cause: A second replica pod remained in `ContainerCreating` indefinitely after adding a PVC to the Deployment; `kubectl describe pod` showed a `FailedAttachVolume -- Multi-Attach error`, since the `ReadWriteOnce` volume was already attached to the first replica on a different node.

Fix: Reduced the Deployment to a single replica, consistent with the access mode's single-node-writer constraint.

10.6 Local Client Connectivity to the Cluster Endpoint

Cause: `kubectl get nodes` timed out when run from outside the VCN – the public subnet's security list scoped TCP/6443 ingress to the private subnet CIDR only (added for node registration, Section 10.3), not to external client IPs.

Fix: Added a scoped ingress rule for TCP/6443 from the developer's public IP (as a `/32`), applied via Terraform.

11 Testing and Validation

11.1 Network Topology

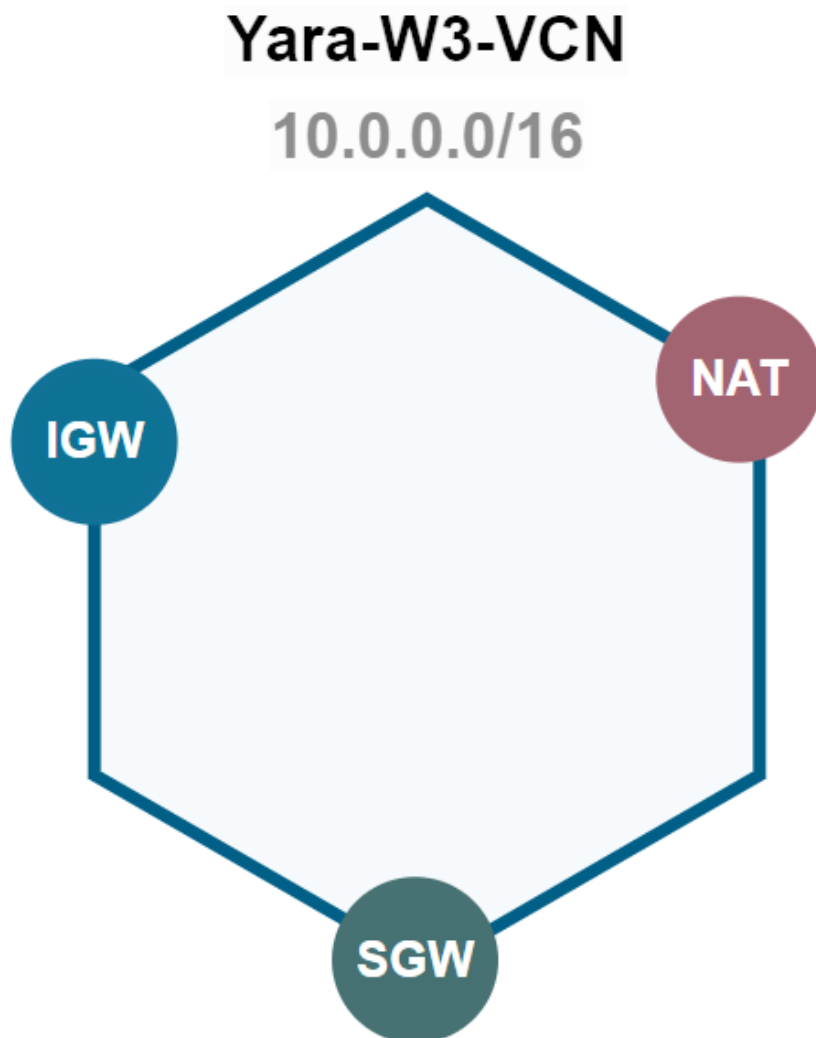


Figure 7: VCN topology – public/private subnet separation

11.2 Cluster and Node Health

Node pool registered with both nodes `Ready`; verified via `kubectl get nodes` (Section 6.3).

11.3 Application and Storage

Check	Result
<code>terraform validate / plan / apply</code>	Clean, no errors
Docker image architecture	<code>linux/arm64</code> , matches node shape
Image pushed to OCIR	Confirmed via repository listing
Pod status	<code>1/1 Running</code>
PVC status	<code>Bound</code> , 50Gi, <code>oci-bv</code>
Persistence test	File survived pod deletion and recreation
LoadBalancer Service	External IP assigned, application reachable over HTTP
Remote state backend	Verified active (Section 9.3)

Table 8: End-to-end validation summary